Guion del video – EFT Desarrollo Backend III (Banco XYZ)

Duración objetivo: 6 min 30 s (rango exigido: 5 a 7 min) · Formato MP4 · Webcam visible durante toda la presentación (recuadro en una esquina) + pantalla con las evidencias.

Antes de grabar (preparación, no aparece en el video)

1. `docker compose down -v && docker compose --profile monitoring up --build -d` y esperar a que todo esté *healthy* (≈ 10 min). Ejecutar una vez `./scripts/demo.sh` para generar tráfico en Grafana.
2. Dejar abiertas estas pestañas/ventanas, en este orden:
   - **P1** – Diagrama de arquitectura (`docs/diagramas/arquitectura.png`)
   - **P2** – Eureka http://localhost:8761
   - **P3** – Terminal 1 con los tokens ya cargados (`WEB`, `MOB`, `ATM`, `ADM`, ver `instrucciones.md` §3)
   - **P4** – Terminal 2 con `docker compose logs -f batch-service notification-service` (limpio con `clear`)
   - **P5** – Tabla de equivalencia legacy vs Spring Batch (`readme.md` §2 o el informe)
   - **P6** – Grafana, dashboard *Banco XYZ – Microservicios*
   - **P7** – Diapositiva o texto con desafíos y mejoras (opcional)
3. Fuente de la terminal grande (≥ 16 pt). Tener los comandos de abajo en un archivo para copiar y pegar.
4. Grabar con Kaltura CaptureSpace (pantalla + webcam), según el instructivo del AVA.

---

1. Presentación (0:00 – 0:20) · Webcam en grande

> *"Hola, soy Emily Rupay. Les presento mi Evaluación Final Transversal de Desarrollo Backend III: la modernización del sistema legacy del Banco XYZ hacia una arquitectura de microservicios en la nube."*

2. Resumen ejecutivo (0:20 – 1:20) · Pantalla P1 (diagrama)

> *"El Banco XYZ funciona hace más de 30 años sobre un mainframe con programas COBOL y scripts Shell, que son caros de mantener, difíciles de escalar y donde una falla en un módulo afecta a todo el sistema.*
>
> *El **objetivo** del proyecto fue migrar ese sistema a una arquitectura moderna, segura y resiliente. El **alcance** tuvo tres partes:*
>
> ***Primero**, migré los tres procesos batch –el reporte de transacciones diarias, el cálculo de intereses mensuales y los estados de cuenta anuales– a **Spring Batch**.*

> ***Segundo**, implementé el patrón **Backend for Frontend** con tres BFF independientes: web, móvil y cajeros automáticos.*
>
> ***Tercero**, construí los microservicios de **Clientes, Cuentas y Pagos** con Spring Cloud: Config Server, Eureka, un Gateway con HTTPS, seguridad OAuth2 con JWT, Resilience4j y eventos en tiempo real con Kafka. Todo corre en 14 contenedores con Docker Compose y está preparado para desplegarse en AWS."*

(Señalar en el diagrama: canales → Gateway → BFF → microservicios → Kafka/PostgreSQL.)

3. Resultados y comparación con el legacy (1:20 – 4:30)

3.1 Plataforma funcionando (1:20 – 1:40) · P2 Eureka

> *"Aquí en Eureka están registrados todos los servicios: el Gateway, los tres BFF, clientes, cuentas, pagos, notificaciones y el batch. Cada uno toma su configuración del Config Server."*

3.2 Parte 1 – Spring Batch (1:40 – 2:40) · P4 logs + P5 tabla + P3 terminal

> *"Al iniciar, el servicio batch ejecutó los tres jobs con los archivos de `fin_legacy_data`."*

Mostrar en P4 (o ejecutar):

```
docker compose logs batch-service | grep -E "Fin|step=" | tail -15
```

> *"Por ejemplo, el cálculo de intereses leyó 1.000 registros, descartó 950 –entre ellos 347 duplicados, edades fuera de rango y saldos vacíos– y calculó los intereses de 50 cuentas en cuatro particiones en paralelo."*

Pantalla P5 (tabla):

> *"Lo más importante es que los resultados son **idénticos al legacy**: reconstruí el proceso legacy en Shell y awk y lo comparé con Spring Batch: mismos 392 movimientos válidos, mismos totales de créditos y débitos, mismos intereses al centavo. Esta comparación además corre como prueba automática."*

Demostrar la tolerancia a fallos (P3):

```
curl -sk -X POST -H "Authorization: Bearer $ADM" "https://localhost:8443/api/batch/jobs/reporteTransaccionesDiariasJob/executions?fechaProceso=2024-12-31&simularFalloCriticoEnLinea=450&forzar=true"
```

> *"Ahora simulo un fallo crítico en la línea 450. El job queda FAILED... (mostrar P4) ...y a los 20 segundos la política de reejecución automática lo reinicia desde el último commit, sin duplicar datos, y termina COMPLETED. Los fallos transitorios, como una caída momentánea de la base de datos, se reintentan hasta tres veces con espera exponencial."*

3.3 Parte 2 – BFF (2:40 – 3:30) · P3 terminal

```
        curl -sk -o /dev/null -w "web: %{size_download} bytes\n"   -H "Authorization: Bear
er $WEB" https://localhost:8443/web/accounts/101/summary
        curl -sk -o /dev/null -w "movil: %{size_download} bytes\n" -H "Authorization: Bear
er $MOB" https://localhost:8443/mobile/accounts/101/summary
        curl -sk -o /dev/null -w "%{http_code}\n" -H "Authorization: Bearer $WEB" https://
localhost:8443/mobile/accounts/101/summary
```

> *"La misma cuenta: el BFF web entrega el historial completo con totales; el BFF móvil entrega solo titular, saldo y cinco movimientos, mucho más liviano, y además usa compresión y ETag. Y si uso el token del canal web en el canal móvil, recibo 403: cada canal tiene su propia autorización."*

```
        KEY=atm-$(date +%s)
        curl -sk -X POST https://localhost:8443/atm/withdrawals -H "Authorization: Bearer
$ATM" -H "X-ATM-ID: ATM-001" -H "Idempotency-Key: $KEY" -H "Content-Type: application/json"
-d '{"cuentaId":101,"monto":20000}'
        # repetir el mismo comando
```

> *"En el cajero, el retiro exige un terminal registrado y una llave de idempotencia: si el cajero pierde la respuesta y reintenta, el dinero **no se descuenta dos veces**. Todo pasa por HTTPS en el Gateway."*

3.4 Parte 3 – Microservicios, eventos y resiliencia (3:30 – 4:30)

```
        curl -sk -X POST https://localhost:8443/api/payments/deposits -H "Authorization: B
earer $ADM" -H "Content-Type: application/json" -d '{"cuentaId":103,"monto":2500000}'
```

> *"Un depósito de 2,5 millones: pagos llama a cuentas, que actualiza el saldo de forma atómica, y publica dos eventos en Kafka: transacción completada y una **alerta de seguridad** por monto alto. (mostrar P4) El servicio de notificaciones los consume en tiempo real."*

```
        docker compose stop accounts-service
        curl -sk -H "Authorization: Bearer $WEB" https://localhost:8443/web/accounts/101/s
ummary
```

> *“Si detengo el servicio de cuentas, el sistema no se cae: el circuit breaker de Resilience4j se abre y el BFF responde con datos degradados y una advertencia; en el cajero se responde ‘servicio no disponible’ y nunca se dispensa dinero sin confirmar. Las transferencias usan el patrón SAGA: si falla el abono, se compensa el cargo.”*

```
docker compose start accounts-service
```

(Mostrar 5 s de Grafana P6: peticiones, latencia y estado de los circuit breakers.)

Comparación con el legacy (decir mientras se ve Grafana):

> *“Frente al legacy: pasamos de un monolito en mainframe a servicios independientes que escalan por separado; de procesos que se reejecutaban a mano a jobs que se reinician solos; de seguridad centralizada a OAuth2 por canal con HTTPS; de un único punto de falla a fallas aisladas con respuestas alternativas; y de no tener visibilidad a métricas, trazas y alertas en tiempo real.”*

4. Desafíos y soluciones (4:30 – 5:30) · Webcam + P7

> *“Los principales desafíos fueron cuatro:*
>
> ***Uno, la calidad de los datos legacy**: cuatro formatos de fecha, montos negativos, tildes, duplicados. Lo resolví separando las reglas de negocio en clases Java puras y probadas, y registrando cada rechazo con su motivo, sin perder ningún registro.*
>
> ***Dos, demostrar que el batch hace lo mismo que el legacy**: reconstruí el proceso legacy en Shell y awk y creé una prueba de equivalencia; también usé redondeo HALF_UP, igual que el ROUNDED de COBOL.*
>
> ***Tres, reintentar sin duplicar**: tanto en el batch como en los pagos. La solución fue la **idempotencia**: limpieza por clave de proceso en el batch, y una llave de idempotencia en cada operación de pago.*
>
> ***Cuatro, la consistencia entre microservicios**, porque cada uno tiene su propia base de datos: usé bloqueo de fila en los saldos, SAGA con compensación en las transferencias y eventos Kafka con confirmación de todas las réplicas.”*

5. Propuestas de mejora y próximos pasos (5:30 – 6:20) · Webcam

> *“Como próximos pasos propongo: - Desplegar en **AWS con ECS Fargate, RDS y MSK**, con escalado automático; ya dejé los scripts y la guía. - Implementar el patrón **Transactional Outbox** para garantizar que cada cambio en la base genere su evento. - Persistir las llaves y clientes del servidor de autorización en **KMS** y base de datos, y agregar **mTLS** entre servicios. - Mover los archivos batch a **S3** y programar los jobs con*

> ***EventBridge**, ejecutando los particionados en varios nodos. - Agregar pruebas de carga y contrato, y un pipeline de **entrega continua** que despliegue automáticamente."*

6. Cierre (6:20 – 6:30) · Webcam

> *"Con esto el Banco XYZ queda con una plataforma escalable, segura y resiliente, lista para la nube. El código, las instrucciones, la guía de despliegue y el informe técnico están en el repositorio. Muchas gracias."*

Lista de verificación del video

- [ ] Duración entre 5:00 y 7:00
- [ ] Webcam visible durante toda la grabación
- [ ] Se cubren los 4 puntos: resumen ejecutivo · resultados y comparación con legacy · desafíos y soluciones · mejoras y próximos pasos
- [ ] Evidencias de las 3 partes: batch (equivalencia + reinicio), BFF (web/móvil/ATM), microservicios (Eureka, Kafka, resiliencia)
- [ ] Exportado en **MP4** y guardado en la misma carpeta que readme.md, instrucciones.md, despliegue.md y el informe PDF